

03

NOT YOUR COLLEGE • CONTROL FLOW

Control Flow

if/if-else, nested conditions, the if-else-if ladder, the ternary operator, and switch

Every program you have written so far runs top to bottom, one line after another, no matter what. Control flow is what breaks that straight line — it lets your program choose which lines to run based on a condition.

if & if-else

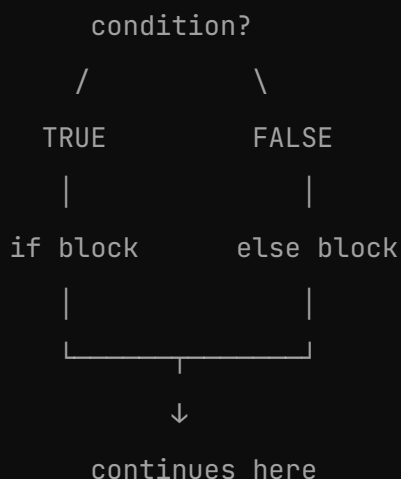
DEFINITION

An if statement runs a block of code only when a condition is true. An if-else gives a second block to run when the condition is false.

Agar condition sahi hai to ek kaam, warna doosra kaam.

The condition is any expression that results in `true` or `false`. With a plain `if`, nothing happens when the condition is false — `if-else` covers that case too.

Only one of the two blocks ever runs.



SYNTAX

SYNTAX

```
if (condition) {  
    // runs when condition is true  
}  
else {  
    // runs when condition is false  
}
```

EXAMPLE

MAIN.CPP

```
int age = 20;  
  
if (age ≥ 18) {  
    cout << "Adult";  
}  
else {  
    cout << "Minor";  
}
```



The condition inside `if ( )` always evaluates to a `bool`. Anything non-zero counts as `true`, and `0` counts as `false` — which is why you can write `if (x)` instead of `if (x ≠ 0)`.

Nested Conditions

DEFINITION

A nested condition is an **if** placed inside another **if** or **else** block.

Ek condition ke andar doosri condition.

The inner condition is only checked when the outer one is already true. Use it when a decision depends on **more than one level** of checking. Too much nesting becomes hard to read — an if-else ladder is often cleaner.

SYNTAX

SYNTAX

```
if (outerCondition) {  
    if (innerCondition) {  
        // both are true  
    }  
}
```

EXAMPLE

MAIN.CPP

```
if (num > 0) {  
    if (num % 2 == 0) {  
        cout << "Positive even";  
    }  
}
```

```
num > 0 ?
|
YES — num % 2 == 0 ?
|           |
|           YES → "Positive even"
|           |
|           NO  → (nothing prints)
NO  → (nothing prints)
```



Both conditions have to be true for the inner block to run. If `num > 0` is false, the inner check is **never even looked at** — the program skips straight past it.



This exact example could also be written as one condition: `if (num > 0 && num % 2 == 0)`. Nesting and `&&` often do the same job — nesting reads better when each level has its own separate logic to run.

A plain if-else only ever picks between two paths. Most real decisions have more than two outcomes — a grade has five bands, not two. This part covers the three tools built for exactly that.

if-else-if Ladder

DEFINITION

An if-else-if ladder **chains multiple conditions**, checking them one after another until one is true.

Ek ke baad ek condition check hoti hai, top se bottom.

Conditions are tested from **top to bottom**. The first true condition runs its block, and the rest are skipped. If none match, the final **else** runs. Use it when you have many different ranges or cases to check.

```
marks ≥ 90 ? — NO → marks ≥ 75 ? — NO → else
    |                      |                      |
    YES                   YES                   |
    |                      |                      |
    "A"                   "B"                   "C"
```

stops at the first true – everything below it is skipped

SYNTAX

SYNTAX

```
if (condition1) {
    // block 1
}
else if (condition2) {
    // block 2
}
else {
    // runs if none matched
}
```

EXAMPLE

MAIN.CPP

```
if (marks ≥ 90) {  
    cout << "A";  
}  
else if (marks ≥ 75) {  
    cout << "B";  
}  
else {  
    cout << "C";  
}
```

⚠ Order matters. If `marks = 95` reached a check for `marks ≥ 75` first, it would print `"B"` and stop — the ladder never even sees the `90` check below it. Always put the **more specific or higher** condition first.

Ternary Operator

DEFINITION

The ternary operator `? :` is a short, one-line form of if-else that **returns a value**.

if-else ka chhota one-line version.

It is the only C++ operator that takes three parts: a condition, a value if true, and a value if false. Best for a simple choice between two values — for anything longer, a normal if-else stays more readable.

SYNTAX

SYNTAX

```
condition ? valueIfTrue : valueIfFalse;
```

EXAMPLE

MAIN.CPP

```
int age = 20;

string result = (age ≥ 18) ? "Adult" : "Minor";
cout << result;    // Adult
```



Read it left to right: "**is the condition true? if yes, use this value — if no, use that one.**" The whole expression evaluates to a single value, which is why it can sit directly inside an assignment like `result = ...`.

IF-ELSE VS TERNARY

IF-ELSE

Runs blocks of statements

Takes several lines

Good for complex logic

TERNARY

Returns a single value

Fits in one line

Good for simple choices

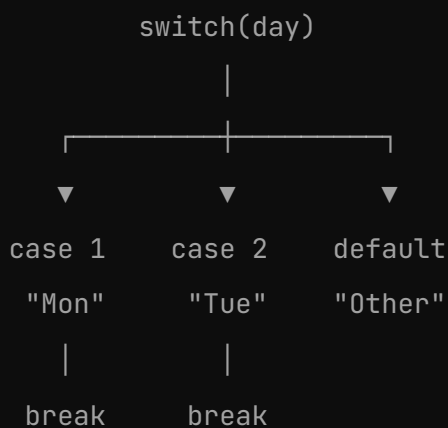
switch Statement

DEFINITION

A switch statement compares **one value** against a list of case labels and runs the matching one.

Ek value ko kai cases se match karke sahi block chalata hai.

It is a cleaner replacement for a long if-else ladder when you are checking one variable against fixed values. Each case usually ends with **break**. If no case matches, the optional **default** block runs.



a case matches, then break jumps straight out

SYNTAX

SYNTAX

```
switch (variable) {  
    case value1:  
        // ...  
        break;  
    case value2:  
        // ...  
        break;  
    default:  
        // no match  
}
```

EXAMPLE

MAIN.CPP

```
int day = 2;  
  
switch (day) {  
    case 1: cout << "Mon"; break;  
    case 2: cout << "Tue"; break;  
    default: cout << "Other";  
}
```

FALL-THROUGH

If you forget `break`, control does not stop — it keeps running the next cases too, until it hits a `break` or the end. This is called **fall-through**. It is usually a bug, but can be used on purpose to let several cases share the same code.

FALL-THROUGH (ON PURPOSE)

```
switch (grade) {  
    case 'A':  
    case 'B': cout << "Pass"; break;    // A and B share this  
    case 'F': cout << "Fail"; break;  
}
```

⚠ Missing a `break` by accident is one of the most common switch bugs — the code still compiles, it just silently runs cases you did not intend to. Get in the habit of typing `break;` the moment you type `case`.

SWITCH VS IF-ELSE LADDER

SWITCH	IF-ELSE LADDER
Checks one variable against fixed values	Checks any conditions or ranges
Cannot check ranges like <code>> 50</code>	Can check ranges freely
Needs break to avoid fall-through	No fall-through
Works with int, char, enum	Works with any condition

SUMMARY

- if / if-else runs a block based on a true/false condition; only one side runs.
- Nested conditions put one check inside another for multi-level decisions.
- if-else-if ladder checks conditions top to bottom and stops at the first true one.
- Ternary `? :` is a one-line if-else that returns a value.
- switch matches a value to cases; always add break to avoid fall-through.